

aliases: - 24-string-methods

字符串方法：查找 / 替换 / 拆分 / 合并

前置：已掌握字符串定义和切片 目标：掌握 Python 最常用的 12 个字符串方法 用时：约 12 分钟

相关笔记：[切片操作](#) · [列表深入操作](#)

概览：字符串方法分类速查

类别	方法	作用
查找	<code>find()</code> / <code>index()</code>	找子串位置
判断	<code>startswith()</code> / <code>endswith()</code>	判断开头/结尾
判断	<code>isalpha()</code> / <code>isdigit()</code> / <code>isalnum()</code>	判断字符类型
修改	<code>replace()</code>	替换
修改	<code>upper()</code> / <code>lower()</code> / <code>title()</code> / <code>capitalize()</code> / <code>swapcase()</code>	大小写转换
修改	<code>strip()</code> / <code>lstrip()</code> / <code>rstrip()</code>	去除两端 / 左端 / 右端空白
修改	<code>center()</code> / <code>ljust()</code> / <code>rjust()</code>	对齐与填充
拆分合并	<code>split()</code> / <code>join()</code>	拆分成列表 / 合并成字符串

一、查找类

find() vs index()

```
text = "Hello, welcome to Python world"

# find: 找到返回起始索引, 找不到返回 -1
print(text.find("welcome"))    # 7 (w 的位置)
print(text.find("Java"))       # -1 (没找到)

# index: 找到返回起始索引, 找不到抛异常
print(text.index("welcome"))    # 7
# print(text.index("Java"))     # ❌ ValueError!
```

用 `find` 还是 `index` ? —— 不确定是否存在时用 `find`, 确定一定存在时用 `index`。

startswith() / endswith()

```
filename = "report_2024.pdf"

print(filename.startswith("report")) # True
print(filename.endswith(".pdf"))     # True
print(filename.endswith(".txt"))     # False

# 实用: 批量处理文件
files = ["data_01.csv", "data_02.csv", "photo.png"]
csv_files = [f for f in files if f.endswith(".csv")]
print(csv_files) # ['data_01.csv', 'data_02.csv']
```

isalpha() / isdigit() / isalnum()

```
# isalpha(): 是否全是字母
print("abc".isalpha())    # True
print("abc123".isalpha()) # False (有数字)

# isdigit(): 是否全是数字
print("123".isdigit())    # True
print("abc".isdigit())    # False

# isalnum(): 是否只有字母和数字 (没有特殊符号)
print("abc123".isalnum()) # True
```

```
print("abc 123".isalnum()) # False (有空格)
print("hello!".isalnum()) # False (有感叹号)
```

实用场景：校验用户输入

```
username = "张三123"
if username.isalnum():
    print("用户名合法")
else:
    print("只能包含字母和数字")

age = input("请输入年龄：")
if age.isdigit():
    print(f"年龄：{int(age)}")
else:
    print("请输入数字")
```

二、修改类

replace() — 替换

```
text = "I like Python, Python is great"

# 全部替换
print(text.replace("Python", "Java"))
# "I like Java, Java is great"

# 指定替换次数
print(text.replace("Python", "Java", 1))
# "I like Java, Python is great"

# 删除字符（替换成空字符串）
print(text.replace(" ", ""))
# "IlikePython,Pythonisgreat"
```

upper() / lower() — 大小写转换

```
text = "Hello World"

print(text.upper())    # "HELLO WORLD"
print(text.lower())    # "hello world"

# 实用：不区分大小写的比较
input_color = "Red"
if input_color.lower() == "red":
    print("匹配")
```

capitalize() / swapcase() / title() — 其他大小写方法

```
text = "hello WORLD"

# capitalize: 句首大写，其余全小写
print(text.capitalize()) # "Hello world"

# title: 每个单词首字母大写
print(text.title())      # "Hello World"

# swapcase: 大写↔小写互换（你文件实验里用过的）
print(text.swapcase())   # "HELLO world"
```

方法	hello WORLD 的结果	规则
upper()	"HELLO WORLD"	全大写
lower()	"hello world"	全小写
capitalize()	"Hello world"	首字母大写，其余全小写
title()	"Hello World"	每个单词首字母大写
swapcase()	"HELLO world"	大小写翻转

strip() — 去除两端空白

```
user_input = "  hello  "

print(repr(user_input.strip()))    # 'hello' — 去两端空格
print(repr(user_input.lstrip()))   # 'hello' — 去左端
print(repr(user_input.rstrip()))   # '  hello' — 去右端


# 实用：清理用户输入
name = input("请输入姓名：").strip()
if name: # 输入全是空格 → 空字符串 → False
    print(f"你好，{name}")
else:
    print("姓名不能为空")
```

三、拆分与合并

split() / join() — 最常用的一对

```
# split: 字符串 → 列表
csv_line = "张三,25,南昌"
fields = csv_line.split(",")
print(fields)    # ['张三', '25', '南昌']


# 默认按空白字符拆分
words = "hello world python".split()
print(words)     # ['hello', 'world', 'python']


# 限制拆分次数
data = "a,b,c,d,e"
print(data.split(",", 2))    # ['a', 'b', 'c,d,e']


# join: 列表 → 字符串 (split 的反操作)
fields = ['张三', '25', '南昌']
csv_line = ",".join(fields)
print(csv_line)    # "张三,25,南昌"


# join 是字符串的方法，不是列表的方法
# 写法是 "分隔符".join(列表)，不是 列表.join("分隔符")
```

```
words = ["Python", "is", "awesome"]  
sentence = " ".join(words)  
print(sentence)    # "Python is awesome"
```

重要： `join()` 的调用对象是**分隔符字符串**，参数是**列表**：

```
", ".join(["a", "b", "c"])    # "a,b,c"  
" ".join(["a", "b", "c"])    # "a b c"  
"".join(["a", "b", "c"])     # "abc"
```

综合示例：CSV 数据解析

```
# 模拟从文件读到的数据  
raw_data = "  张三, 25, 南昌  \n  李四, 30, 北京  "
```



```
# 清洗流程  
cleaned = raw_data.strip()           # 去两端空白  
lines = cleaned.split("\n")         # 按换行拆成行  
for line in lines:  
    fields = [f.strip() for f in line.split(",")] # 每行按逗号拆分+去空格  
    print(fields)    # ['张三', '25', '南昌'] ['李四', '30', '北京']
```

四、其他实用方法

```
# count: 统计子串出现次数  
text = "apple banana apple cherry apple"  
print(text.count("apple"))    # 3
```



```
# len: 获取字符串长度（不是方法，是函数）  
print(len("Hello"))           # 5
```



```
# in: 判断是否包含（最常用的方式）  
print("apple" in text)        # True  
print("orange" in text)       # False
```

`in` 比 `find()` 更直观, 判断是否存在时优先用 `in`。

实战案例（来自你微信里的 Python 学习文件）

案例1：字符串表示（引号与转义） → 字符串表示.py

```
print('使用单引号定义的字符串')
print("使用双引号定义的字符串")
print("""使用三引号定义的
      多行字符串""")

# 字符串里包含引号的处理
print("let's learn Python")          # 外双内单
print('let\'s learn Python')          # 反斜杠转义
print("How do you spell the word \"Python\"?") # 双引号转义
```

要点：单引号 `'`、双引号 `"`、三引号 `"""` 都可以定义字符串。引号嵌套时用 `\` 转义。

案例2：大小写转换 → 字符串大小写转换.py

```
old_string = 'hello world'
print(old_string.upper())    # HELLO WORLD — 全大写
print(old_string.lower())    # hello world — 全小写
print(old_string.capitalize()) # Hello world — 首字母大写
print(old_string.title())     # Hello World — 每个单词首字母大写
```

案例3：查找与替换 → 字符串的查找与替换.py

```
# find() 查找子串位置
string = "Pythontn"
print(string.find('t'))      # 2 — 从0开始找't'首次出现
print(string.find('t', 5))   # 6 — 从索引5开始找

# replace() 替换
```

```
string = "All things Are difficult before they Are easy Are good"
print(string.replace("Are", "are"))      # 全部替换
print(string.replace("Are", "are", 2))   # 只替换前2次
```

案例4：分割与拼接 → 字符串的分割与拼接.py

```
# split() 分割
string = "The more efforts you make,the more fortune you get"
print(string.split())      # 默认按空格分割
print(string.split('m'))   # 按'm'分割
print(string.split('e', 2)) # 按'e'分割，只分2次

# join() 拼接 + 加号拼接
print('*'.join('Python'))  # P*y*t*t*h*o*n
print('Py' + 'thon')       # Python — +号拼接
```

案例5：字符串对齐 → 字符串对齐.py

```
sentence = 'hello world'
print(sentence.center(13, '-')) # -hello world- 居中，-补齐
print(sentence.ljust(13, '*'))  # hello world** 左对齐，*补齐
print(sentence.rjust(13, '%'))  # %%hello world 右对齐，%补齐
```

案例6：删除指定字符 → 删除字符串的指定字符.py

```
old = ' Life is short,Use Python! '
print(f"strip:{old.strip()}显示") # 去两端空格
print(f"lstrip:{old.lstrip()}显示") # 去左端空格
print(f"rstrip:{old.rstrip()}显示") # 去右端空格
```

总结速记

查找用 `find`（找不到不报错），判断存在用 `in`
去空格用 `strip`，大小写用 `lower/upper`
拆分用 `split`，合并用 `join`（分隔符.`join(列表)`）
判断类型用 `isdigit/isalpha/isalnum`

实战场景：

引号与转义 → 单/双/三引号、`\'` `\"`
大小写 → `upper()` / `lower()` / `capitalize()` / `title()`
查找替换 → `find()` / `replace()`
分割拼接 → `split()` / `join()` / `+`
对齐 → `center()` / `ljust()` / `rjust()`
去空格 → `strip()` / `lstrip()` / `rstrip()`

备考相关

- [[EXAM_PREP/day02/00_今日任务]] — Day 2 字符串与组合数据类型